UsbLogSink — Integration Plan

Goal: Stream telemetry/log lines (hundreds per second) into a `.txt` / `.log` file on a USB drive mounted at `/mnt/media_rw/<VOLUME>/`, surviving Android multi-user switches without vold ripping the mount out from under an open file descriptor.

Deliverable in this package: `UsbLogSink.kt` — a self-contained, thread-safe sink class. This document explains how to wire it into the existing project. No changes to `UsbLogSink.kt` should be needed beyond the package declaration.

---

1. Architecture at a glance

```
Any thread (log producers, 100s/sec)
        │  sink.offer(line)          ← non-blocking, never touches file
        ▼
ArrayBlockingQueue<String>  (capacity 100k — this IS the switch buffer)
        │  drainTo() in batches of 512
        ▼
Single writer thread ("UsbLogSink-writer")  ← sole owner of the file
        │  one write() syscall per batch, flush every 500ms / 64KB
        ▼
/mnt/media_rw/<VOL>/logs/current.log   (rotates at 10 MB → log-<ts>.log)
```

State machine (writer thread)

State	Meaning	File open?	Queue
`NO_MEDIA`	No USB / open failed / write error	No	Absorbs (drops when full, counted)
`WRITING`	Normal streaming	Yes	Drained in batches
`SUSPEND_REQUESTED`	User switch (or eject) incoming	Closing	Absorbs
`SUSPENDED`	Switch in progress	No	Absorbs — this is the “buffer during switch”
`STOPPED`	Terminal	No	—

The user-switch contract (the important part)

1. `IUserSwitchObserver.onUserSwitching(newUserId, reply)` fires → call `sink.suspendForUserSwitch(newUserId, reply)`.

2. The writer thread drains pending entries, `flush()` → `fsync()` → `close()`, **then** fires `reply.sendResult(null)`.
3. ActivityManager blocks the switch until that reply, so **vold can never remount the volume while we hold an fd**. Cost: milliseconds.
4. During the switch, producers keep calling `offer()`; entries pile up in the queue (no second buffer exists — the queue is the buffer).
5. `onUserSwitchComplete(newUserId)` fires → call `sink.resumeAfterUserSwitch(newUserId)`. Writer reopens `current.log` (append mode), writes a `=== resumed after switch to user N ===` marker, and drains everything accumulated.

Do NOT send the reply yourself before calling `suspendForUserSwitch`, and do not add a timeout wrapper that acks early — that reintroduces the vold race the whole design exists to prevent.

---

2. Files to add

File	Destination (adjust to project layout)
`UsbLogSink.kt`	`<module>/src/main/java/com/samsung/teachingboard/telemetry/usbsink/UsbLogSink.kt`

Update the `package` line if the project uses a different namespace.

Dependencies: none beyond the Android framework. Uses `android.system.Os` (fsync), `android.os.IRemoteCallback` (hidden API — fine here because the host app is platform-signed and already implements `IUserSwitchObserver` via AIDL; reuse the same AIDL/stub approach already present in the project for the `IRemoteCallback` type).

---

3. Wiring — 4 integration points

3.1 Create + start the sink (once, in the long-lived service)

In the existing telemetry/observer service (the one that registers `IUserSwitchObserver`):

```
val usbLogSink = UsbLogSink()   // defaults are production-sane; see §5
usbLogSink.start()
```

Hold a single instance for the process lifetime. Do not create per-user or per-mount instances.

3.2 User switch observer (already exists in the project)

Inside the existing `IUserSwitchObserver.Stub` implementation:

```
override fun onUserSwitching(newUserId: Int, reply: IRemoteCallback?) {
    usbLogSink.suspendForUserSwitch(newUserId, reply)
    // Do NOT call reply.sendResult() here — the sink does it after close.
```

```
}

override fun onUserSwitchComplete(newUserId: Int) {
    usbLogSink.resumeAfterUserSwitch(newUserId)
}
```

If the existing observer already sends the reply for other reasons, refactor so the reply is sent exactly once, and only after the sink's close completes (chain callbacks if multiple components need to gate the switch).

3.3 Media availability

Hook the project's existing `ACTION_MEDIA_MOUNTED` / startup-scan path:

```
// On mount / startup scan finding the volume:
usbLogSink.onMediaMounted(File("/mnt/media_rw/$volumeUuid"))

// On ACTION_MEDIA_EJECT or ACTION_MEDIA_UNMOUNTED for that volume:
usbLogSink.onMediaEjected()
```

Notes:

- Pass the **raw** `/mnt/media_rw/...` path (the app already has access), not `/storage/...`.
- `onMediaMounted` is idempotent; safe to call again after a switch if the scan re-fires.
- Eject is handled defensively twice: the eject callback closes proactively, and any `IOException` mid-write also demotes to `NO_MEDIA` (batch counted as dropped). Both paths are already inside `UsbLogSink`.

3.4 Producers

Wherever log lines are generated (collectors, logcat reader thread, etc.):

```
usbLogSink.offer(formattedLine)   // no trailing '\n'; any thread; never blocks
```

Format lines **before** offering (timestamp, tag, etc.) so the writer thread does zero per-entry work beyond concatenation.

3.5 Shutdown

In the service's `onDestroy` / teardown path:

```
usbLogSink.stop()   // flush + fsync + close + join (≤5s)
```

---

4. Behavioral guarantees & non-guarantees

Guaranteed:

- File is fully closed (flushed + fsynced) before the user switch proceeds.

- Single writer thread → no file-level locking anywhere.
- One write syscall per batch (≤512 entries); flush at most every 500 ms or 64 KB; fsync only at rotation/suspend/eject/stop. This is what makes 100s/sec cheap on slow USB flash.
- Rotation: `current.log` → `log-<yyyyMMdd-HHmmss>.log` at 10 MB, with data fsync before rename and directory fsync after (rename durability). Downstream consumers (the cross-user sync ContentProvider) should continue to expose only rotated files, never `current.log`.
- Drop accounting: if the queue fills (only plausible if media is absent for minutes) or a batch dies on `IOException`, drops are counted and a `=== N entries dropped ===` marker is written on next open.

Not guaranteed / by design:

- Entries queued but not yet written are lost on process death (in-memory queue). If crash durability matters later, lower `flushIntervalMs` — do not add per-entry fsync.
- Ordering across a drop event has a gap (that's what the marker is for).

---

5. Tunables (constructor `Config`)

Param	Default	When to change
`queueCapacity`	100 000	Raise if switches ever exceed ~3 min at your log rate (they won't)
`batchSize`	512	Rarely
`streamBufferBytes`	128 KB	Rarely
`flushIntervalMs`	500 ms	Lower for tighter crash durability, higher to reduce USB wear
`flushThresholdBytes`	64 KB	With flush interval
`rotateAtBytes`	10 MB	Per fleet log-retention policy
`pollTimeoutMs`	200 ms	Bounds suspend-request latency; leave as is
`currentFileName` / `logSubDir`	`current.log` / `logs`	Match existing fleet layout

---

6. Test checklist (SQA + dev)

1. **Sustained throughput:** feed 1 000 lines/sec for 5 min → no drops (`droppedCount` marker absent), CPU of writer thread low, file contents complete and ordered.
2. **User switch mid-stream:** stream at 500/sec, trigger switch. Verify (a) switch is not visibly delayed, (b) no vold errors / `EIO` in logcat, (c) file contains pre-switch lines, then the `=== resumed after switch to`

`user N ===` marker, then post-switch lines with **no gap**.

3. **Rapid double switch:** switch A→B→A quickly. No crash, single reply per `onUserSwitching`, markers correct.
4. **USB yank mid-write:** pull the drive while streaming. Process must not crash; sink lands in `NO_MEDIA`; re-insert → `onMediaMounted` → writing resumes with drop marker.
5. **Eject during a switch:** unplug while `SUSPENDED`. On `onUserSwitchComplete`, sink must go `NO_MEDIA` (not attempt open); replug resumes.
6. **Rotation under load:** set `rotateAtBytes = 1 MB` in a debug build, stream until several rotations occur; verify rotated files are complete, `current.log` restarts, no interleaving corruption.
7. **Switch while nothing mounted:** trigger a user switch with no USB present → reply must still be sent immediately (switch not blocked).
8. **Samsung framework check:** confirm on the actual teaching-board firmware that `onUserSwitching` delivers a non-null `IRemoteCallback` and that AMS waits on it (Samsung's AMS diverges from AOSP in places — verify with logs around the switch, don't assume).

---

7. Known sharp edges for the implementer

- `IRemoteCallback` reply must be sent **exactly once** per `onUserSwitching`. `UsbLogSink` owns this once `suspendForUserSwitch` is called — never ack it elsewhere.
- Keep the close path fast. Do not add extra work (uploads, scans) between `onUserSwitching` and the reply; AMS timeouts on slow observers will log warnings and may proceed anyway, reintroducing the race.
- Do not wrap `offer()` call sites in synchronization — it's already thread-safe and non-blocking.
- Do not open the raw path from any other component while the sink is running; single-owner is a core invariant.
- The writer thread reopens lazily: after resume/rotation the file opens on the first batch, not eagerly. This is intentional (handles resume-with-no-media cleanly).